SIMATS ENGINEERING
ASSESSMENT TOOL 3

COURSE NAME: Fundamentals of Data Science **COURSE CODE: DSA0402**

COURSE OUTCOME COVERED

CO2: Apply data pre-processing and visualization techniques using Python libraries for effective data handling. (BL4)

QUESTIONS: 05 **Total Marks:100**

Annexure A

ANALYTICAL PROBLEM SOLVING

Code of Conduct:

*I **Jannathul Firdaus. N (192573025)** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Jannathul Firdaus. N

Annexure A

Analytical Problem Solving

1. Online Food Delivery Data Analysis
An online food delivery company wants to analyze order data to improve delivery performance and customer satisfaction. Explain how:
A. **Pandas** is used to read, clean, and filter order data
B. **Data preprocessing** is applied to handle missing delivery time, duplicate orders, and incorrect prices
C. **Matplotlib or Seaborn** is used to visualize delivery delays, top restaurants, and customer ratings

2. Bank Loan Approval Analysis
A bank wants to study customer loan applications and identify patterns related to loan approval and rejection. Explain how:
A. **Pandas DataFrame operations** are used to select income, loan amount, credit score, and approval status
B. **Preprocessing techniques** are used to handle missing income values, encode gender/employment type, and normalize loan amount
C. **Visualization** is used to compare approved and rejected applications based on income, credit score, and loan amount

3. Smart Energy Consumption Monitoring
A smart home system collects daily electricity consumption data from different appliances such as AC, fan, light, and refrigerator. Explain how:
A. **Pandas** is used to group electricity usage by appliance and date
B. **Data preprocessing** is used to remove abnormal readings, fill missing values, and calculate total power consumption
C. **Line charts and bar charts** are used to identify high-energy appliances and daily usage trends

4. Online Learning Platform Performance Analysis
An online learning platform wants to analyze student activity such as login time, video completion, quiz score, and assignment submission. Explain how:
A. **Pandas** is used to sort, filter, and rank students based on quiz scores and completion percentage
B. **Preprocessing** is applied to handle missing quiz scores, convert date formats, and create new columns such as performance category
C. **Seaborn and Matplotlib** are used to visualize student engagement, score distribution, and course completion status

5. Public Transport Passenger Data Analysis
A city transport department wants to analyze bus passenger data to improve route planning and reduce overcrowding. Explain how:
A. **Pandas** is used to analyze passenger count by route, stop, and time slot
B. **Data preprocessing** is used to clean invalid route numbers, handle missing passenger counts, and remove duplicate records
C. **Visualization techniques** are used to show peak travel hours, busiest routes, and passenger flow patterns

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

Question 1 – Online Food Delivery Data Analysis

Objective: assess delivery performance, restaurant service, order prices, and customer satisfaction.

A. Pandas operations

A. Pandas: read the CSV, inspect the schema, select relevant columns, and filter delayed orders or high ratings.

```
import pandas as pd
orders = pd.read_csv("food_delivery_orders.csv")
print(orders.head())
orders.info()
cols = ["order_id", "restaurant", "order_date", "order_price",
"delivery_time_min", "customer_rating"]
orders = orders[cols]
delayed = orders[orders["delivery_time_min"] > 45]
high_rated = orders[orders["customer_rating"] >= 4.5]
```

B. Data preprocessing

B. Preprocessing: parse dates and numeric fields, remove exact duplicate rows, investigate repeated order IDs, handle missing delivery time without treating it as zero, and flag negative or otherwise invalid prices for verification.

```
orders["order_date"] = pd.to_datetime(orders["order_date"], errors="coerce")
for c in ["delivery_time_min", "order_price", "customer_rating"]:
orders[c] = pd.to_numeric(orders[c], errors="coerce")
orders = orders.drop_duplicates()
duplicate_ids = orders[orders.duplicated("order_id", keep=False)]
orders["delivery_time_missing"] = orders["delivery_time_min"].isna()
# Example imputation only when justified by the data:
orders["delivery_time_min"] = orders.groupby("restaurant")[
"delivery_time_min"].transform(lambda s: s.fillna(s.median()))
orders["invalid_price"] = orders["order_price"].isna() | (orders["order_price"] < 0)
```

C. Visualization

C. Visualization: a histogram or box plot displays delivery-time distribution; a bar chart ranks restaurants by average delivery time; and a rating plot compares customer feedback.

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(data=orders, x="delivery_time_min", bins=20, kde=True)
plt.title("Distribution of Delivery Time")
plt.xlabel("Delivery time (minutes)"); plt.ylabel("Orders")
plt.tight_layout(); plt.show()
orders.groupby("restaurant")["delivery_time_min"].mean().nlargest(10).plot(kind="bar")
plt.title("Restaurants with Highest Average Delivery Time")
plt.ylabel("Average delivery time (minutes)")
plt.tight_layout(); plt.show()
sns.boxplot(data=orders, x="restaurant", y="customer_rating")
```

```
plt.title("Customer Ratings by Restaurant")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
```

Expected insights and recommendations

- Measure the percentage of orders exceeding a defined delivery-time target.
- Compare average and median delivery times by restaurant to avoid relying on a single summary.
- Examine whether long delivery times coincide with lower ratings, while considering distance, weather, and order complexity.
- Use findings to improve dispatching and review restaurant-level service; association alone does not establish cause.

Question 2 – Bank Loan Approval Analysis

Objective: compare approved and rejected applications by income, loan amount, credit score, and employment characteristics.

A. Pandas operations

A. Pandas: select the relevant fields and use groupby() to compare approval groups.

```
loans = pd.read_csv("loan_applications.csv")
selected = loans[["income", "loan_amount", "credit_score", "approval_status"]]
summary = selected.groupby("approval_status")[
["income", "loan_amount", "credit_score"]].agg(
["count", "mean", "median", "min", "max"])
approved = selected[selected["approval_status"] == "Approved"]
rejected = selected[selected["approval_status"] == "Rejected"]
print(summary)
```

B. Data preprocessing

B. Preprocessing: investigate missing income before filling it; retain a missingness flag; one-hot encode nominal categories such as gender and employment type; and normalize loan amount if needed. For predictive modelling, fit imputers and scalers on training data only to prevent data leakage.

```
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
loans["income"] = pd.to_numeric(loans["income"], errors="coerce")
loans["income_missing"] = loans["income"].isna()
loans["income"] = loans["income"].fillna(loans["income"].median())
loans = pd.get_dummies(loans, columns=["gender", "employment_type"],
dtype=int)
scaler = MinMaxScaler()
loans["loan_amount_normalized"] = scaler.fit_transform(
loans[["loan_amount"]])
```

C. Visualization

C. Visualization: use box plots for income and credit score by approval status, scatter plots for credit score versus loan amount, and separate bar charts for group averages.

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.boxplot(data=loans, x="approval_status", y="income")
plt.title("Income by Loan Approval Status")
plt.tight_layout(); plt.show()
sns.scatterplot(data=loans, x="credit_score", y="loan_amount",
hue="approval_status")
plt.title("Credit Score and Loan Amount by Approval Status")
plt.tight_layout(); plt.show()
loans.groupby("approval_status")["credit_score"].mean().plot(kind="bar")
plt.title("Mean Credit Score by Approval Status")
plt.ylabel("Credit score"); plt.tight_layout(); plt.show()
```

Expected insights and recommendations

- Compare distributions rather than assuming one variable determines approval.
- Validate observed patterns against lending policy and a larger sample.
- Review fairness, legal requirements, and potential bias in sensitive attributes.
- Evaluate any predictive model for calibration and error rates before operational use.

Question 3 – Smart Energy Consumption Monitoring

Objective: find high-energy appliances and daily electricity-use trends.

A. Pandas operations

A. Pandas: parse timestamps, derive dates, and group energy readings by date and appliance. Sum interval energy in kWh; instantaneous power readings require time-aware integration.

```
energy = pd.read_csv("energy_readings.csv")
energy["timestamp"] = pd.to_datetime(energy["timestamp"], errors="coerce")
energy["date"] = energy["timestamp"].dt.date
daily_appliance = energy.groupby(["date", "appliance"], as_index=False)[
"consumption_kwh"].sum()
daily_total = daily_appliance.groupby("date", as_index=False)[
"consumption_kwh"].sum()
```

B. Data preprocessing

B. Preprocessing: check missing timestamps, negative readings, duplicate meter records, and physically implausible values. Flag abnormal readings using meter specifications; do not automatically delete large readings because they may be genuine. Interpolate only when interval regularity and context justify it.

```
energy = energy.drop_duplicates(subset=["timestamp", "appliance"])
energy["consumption_kwh"] = pd.to_numeric(
energy["consumption_kwh"], errors="coerce")
energy["abnormal_reading"] = energy["consumption_kwh"] < 0
energy.loc[energy["abnormal_reading"], "consumption_kwh"] = float("nan")
energy = energy.sort_values(["appliance", "timestamp"])
energy["consumption_kwh"] = energy.groupby("appliance")[
"consumption_kwh"].transform(lambda s: s.interpolate(limit=2))
daily_appliance = energy.groupby(
[energy["timestamp"].dt.date, "appliance"]
)["consumption_kwh"].sum(min_count=1).reset_index()
daily_appliance.columns = ["date", "appliance", "consumption_kwh"]
```

C. Visualization

C. Visualization: line charts show changes in daily consumption; bar charts rank appliances by total kWh; stacked bars can show each appliance's contribution to daily usage.

```
import matplotlib.pyplot as plt
daily_total = daily_appliance.groupby("date", as_index=False)[
"consumption_kwh"].sum(min_count=1).sort_values("date")
plt.plot(daily_total["date"], daily_total["consumption_kwh"], marker="o")
plt.title("Daily Household Electricity Consumption")
plt.xlabel("Date"); plt.ylabel("Consumption (kWh)")
plt.xticks(rotation=45); plt.tight_layout(); plt.show()
daily_appliance.groupby("appliance")["consumption_kwh"].sum().sort_values(
ascending=False).plot(kind="bar")
plt.title("Total Consumption by Appliance")
plt.ylabel("Consumption (kWh)"); plt.tight_layout(); plt.show()
```

Expected insights and recommendations

- Identify appliances with the largest total energy use.
- Use the daily line chart to locate recurring peaks and unusual days.
- Check data coverage before comparing days with different amounts of missing data.
- Recommend appliance scheduling or energy-saving actions based on measured use and household context.

Question 4 – Online Learning Platform Performance Analysis

Objective: understand student engagement and achievement from login activity, video completion, quiz scores, and assignment submission.

A. Pandas operations

A. Pandas: sort students by quiz score and completion percentage, rank students, and filter records needing follow-up.

```
students = pd.read_csv("learning_activity.csv")
students = students.sort_values(
```

```
["quiz_score", "video_completion_pct"], ascending=[False, False])
students["quiz_rank"] = students["quiz_score"].rank(
method="dense", ascending=False)
low_completion = students[students["video_completion_pct"] < 60]
missing_assignments = students[students["assignment_submitted"] == False]
print(students.head(10))
```

B. Data preprocessing

B. Preprocessing: determine why quiz scores are missing before imputation, convert date fields using to_datetime(), and create performance categories based on documented course thresholds.

```
import pandas as pd
students["quiz_score"] = pd.to_numeric(students["quiz_score"], errors="coerce")
students["quiz_score_missing"] = students["quiz_score"].isna()
students["login_time"] = pd.to_datetime(students["login_time"], errors="coerce")
students["assignment_date"] = pd.to_datetime(
students["assignment_date"], errors="coerce")
def category(score):
if pd.isna(score): return "Not recorded"
if score >= 85: return "High"
if score >= 60: return "Moderate"
return "Needs support"
students["performance_category"] = students["quiz_score"].apply(category)
```

C. Visualization

C. Visualization: a histogram displays score distribution; a scatter plot compares completion percentage with quiz score; and a bar chart shows the number of students in each performance category.

```
import matplotlib.pyplot as plt
import seaborn as sns
sns.histplot(data=students, x="quiz_score", bins=10, kde=True)
plt.title("Distribution of Student Quiz Scores")
plt.xlabel("Quiz score"); plt.ylabel("Students")
plt.tight_layout(); plt.show()
sns.scatterplot(data=students, x="video_completion_pct", y="quiz_score",
hue="performance_category")
plt.title("Video Completion and Quiz Score")
plt.tight_layout(); plt.show()
students["performance_category"].value_counts().plot(kind="bar")
plt.title("Students by Performance Category")
plt.ylabel("Number of students"); plt.tight_layout(); plt.show()
```

Expected insights and recommendations

- Identify learners who may benefit from timely, supportive intervention.
- Compare completion patterns with quiz outcomes without claiming causation.
- Use score bands aligned with course policy and make missing-score reasons explicit.

- Interpret engagement carefully because platform activity may not capture offline study or accessibility needs.

Question 5 – Public Transport Passenger Data Analysis

Objective: analyze passenger counts by route, stop, and time slot to improve route planning and reduce overcrowding.

A. Pandas operations

A. Pandas: parse timestamps, derive hour-of-day, group passenger counts by route and stop, and create a route-by-hour pivot table.

```
passengers = pd.read_csv("bus_passengers.csv")
passengers["timestamp"] = pd.to_datetime(
passengers["timestamp"], errors="coerce")
passengers["hour"] = passengers["timestamp"].dt.hour
by_route = passengers.groupby("route_number")["passenger_count"].sum()
by_stop = passengers.groupby("stop_name")["passenger_count"].sum()
by_hour = passengers.groupby("hour")["passenger_count"].sum()
route_hour = passengers.pivot_table(index="route_number", columns="hour",
values="passenger_count", aggfunc="sum", fill_value=0)
```

B. Data preprocessing

B. Preprocessing: standardize route numbers and validate them against the official route list. Convert passenger counts to numeric, flag negative counts, distinguish missing counts from true zero counts, and remove duplicates only using a key that represents a genuinely unique observation.

```
passengers["route_number"] = passengers["route_number"].astype(
"string").str.strip()
passengers["passenger_count"] = pd.to_numeric(
passengers["passenger_count"], errors="coerce")
valid_routes = {"01", "02", "03", "10", "12"} # replace with official list
passengers["invalid_route"] = ~passengers["route_number"].isin(valid_routes)
passengers.loc[passengers["passenger_count"] < 0,
"passenger_count"] = float("nan")
passengers["count_missing"] = passengers["passenger_count"].isna()
# Adapt key to actual data granularity; include bus ID when available
passengers = passengers.drop_duplicates(
subset=["timestamp", "route_number", "stop_name", "bus_id"])
```

C. Visualization

C. Visualization: a line chart displays passenger demand by hour, a bar chart ranks busiest routes, and a heatmap reveals route-specific peak hours. Compare passenger counts with vehicle capacity if overcrowding is the target.

```
import matplotlib.pyplot as plt
import seaborn as sns
by_hour.sort_index().plot(kind="line", marker="o")
plt.title("Passenger Demand by Hour")
```

```
plt.xlabel("Hour of day"); plt.ylabel("Passenger count")
plt.xticks(range(24)); plt.tight_layout(); plt.show()
by_route.nlargest(10).plot(kind="bar")
plt.title("10 Busiest Bus Routes")
plt.ylabel("Passenger count"); plt.tight_layout(); plt.show()
sns.heatmap(route_hour)
plt.title("Passenger Count by Route and Hour")
plt.xlabel("Hour of day"); plt.ylabel("Route number")
plt.tight_layout(); plt.show()
```

Expected insights and recommendations

- Identify peak hours and routes with the greatest passenger demand.
- Use stop-level summaries to identify locations where passenger flows concentrate.
- Do not treat a missing sensor reading as zero passengers.
- Consider vehicle capacity, route length, operating cost, and data coverage before changing service frequency.